

NiceGUI 生命周期事件（Lifecycle events）全面详细解析

NiceGUI 的生命周期事件（Lifecycle events）是框架级别的事件机制，用于监听和响应应用运行过程中的核心阶段（如启动、关闭、客户端连接 / 断开等），开发者可通过注册回调函数（协程或普通函数），在这些关键节点执行自定义逻辑（如初始化资源、清理数据、记录客户端状态等）。相比通用的 `Event` 类，生命周期事件是框架预定义的“全局钩子”，聚焦于应用和客户端的生命周期管理。

一、核心生命周期事件列表

NiceGUI 提供 6 个核心生命周期事件，覆盖应用、客户端、异常处理三大维度，每个事件对应明确的触发时机和参数规则，具体如下：

事件名	触发时机	可选参数	版本说明	核心用途
<code>app.on_startup</code>	NiceGUI 应用启动 / 重启时	无	-	应用初始化（如连接数据库、加载配置、启动后台任务）
<code>app.on_shutdown</code>	NiceGUI 应用关闭 / 重启时	无	-	资源清理（如关闭数据库连接、保存临时数据、终止后台任务）
<code>app.on_connect</code>	每个客户端（浏览器 / 设备）连接时（包括重连）	<code>nicegui.Client</code> （客户端对象）	-	记录客户端连接状态、初始化客户端专属数据、权限校验
<code>app.on_disconnect</code>	每个客户端断开连接时（包括重连过程中的临时断开）	<code>nicegui.Client</code> （客户端对象）	3.0.0 版本优化参数逻辑	临时清理客户端资源（如暂停该客户端的后台任务）
<code>app.on_delete</code>	客户端被永久删除时（未重连且框架清理该客户端资源）	<code>nicegui.Client</code> （客户端对象）	3.0.0 新增	彻底清理客户端相关资源（如删除该客户端的临时文件、释放专属内存）
<code>app.on_exception</code>	应用运行过程中抛出未捕获异常时	<code>exception</code> （异常对象）	-	全局异常捕获、日志记录、错误告警（如推送异常信息到监控系统）

关键补充规则

1. 回调函数支持：所有生命周期事件的回调可以是**普通函数**（同步）或**协程函数**（`async def` 定义，异步），框架会自动适配执行；
2. 任务自动取消：当应用关闭 / 重启时，所有仍在执行的任务（包括生命周期事件回调中启动的任务）会被框架自动取消，无需手动终止；
3. 触发范围：`on_startup / on_shutdown` 是**应用级**（全局仅触发一次），`on_connect / on_disconnect / on_delete` 是**客户端级**（每个客户端触发一次），`on_exception` 是**全局级**（任意位置抛异常均触发）。

二、核心事件详解与使用示例

1. 应用级事件：on_startup & on_shutdown

触发逻辑

- `on_startup`：在 `ui.run()` 执行后、应用开始监听请求前触发；若应用重启（如通过 `ui.run(reload=True)` 热重载），会再次触发；
- `on_shutdown`：在应用停止监听请求前触发（如手动终止进程、热重载重启时），优先于任务取消逻辑执行。

示例：初始化 / 清理数据库连接

```
from nicegui import app, ui
import asyncio
import aiofiles # 示例：异步文件操作，模拟数据库连接

# 全局数据库连接对象
db_conn = None

# 应用启动时初始化连接
async def init_db():
    global db_conn
    print('应用启动，初始化数据库连接...')
    # 模拟异步连接数据库
    await asyncio.sleep(0.5)
    db_conn = 'mock_db_connection' # 模拟连接对象
    print('数据库连接完成')
app.on_startup(init_db)

# 应用关闭时清理连接
def close_db():
    global db_conn
    if db_conn:
        print('应用关闭，释放数据库连接...')
        db_conn = None # 模拟关闭连接
        print('数据库连接已释放')
app.on_shutdown(close_db)

ui.label('应用已启动，数据库连接状态：正常')
ui.run(reload=True) # 热重载时会触发 shutdown + startup
```

2. 客户端级事件：on_connect & on_disconnect & on_delete

核心区别（易混淆点）

事件	触发场景	资源状态	适用操作
on_connect	客户端首次连接 / 重连	客户端资源已创建	初始化客户端会话、加载用户配置
on_disconnect	客户端临时断开（如网络波动、页面刷新）	客户端资源仍保留（等待重连）	暂停客户端专属任务、标记“离线中”
on_delete	客户端永久断开（超时未重连，框架清理资源）	客户端资源即将销毁	彻底删除临时文件、释放专属内存

示例：记录客户端连接 / 断开状态

```
from nicegui import app, ui, Client
from datetime import datetime
from typing import Dict

# 存储客户端连接信息：{client.id: 连接时间}
client_connections: Dict[str, datetime] = {}

# 客户端连接时记录
def on_client_connect(client: Client):
    client_connections[client.id] = datetime.now()
    print(f'客户端 {client.id} 连接, 时间: {client_connections[client.id]}')
app.on_connect(on_client_connect)

# 客户端断开时临时标记
def on_client_disconnect(client: Client):
    if client.id in client_connections:
        print(f'客户端 {client.id} 临时断开连接')
app.on_disconnect(on_client_disconnect)

# 客户端被删除时彻底清理
def on_client_delete(client: Client):
    if client.id in client_connections:
        del client_connections[client.id]
        print(f'客户端 {client.id} 已永久删除, 清理连接记录')
app.on_delete(on_client_delete)

# UI 展示当前连接的客户端数量
label = ui.label(f'当前在线客户端数: {len(client_connections)}')
ui.timer(1, lambda: label.set_text(f'当前在线客户端数: {len(client_connections)}'))

ui.run()
```

3. 全局异常事件: on_exception

触发逻辑

- 捕获应用内**所有未手动捕获**的异常（包括 UI 回调、后台任务、生命周期事件回调中的异常）；
- 回调函数可接收 `exception` 参数（异常对象），用于获取异常类型、栈信息等。

示例：全局异常日志记录

```
from nicegui import app, ui
import traceback

# 全局异常处理
def handle_exception(exception: Exception):
    # 记录异常信息到控制台（实际场景可写入日志文件/推送告警）
    print('\n===== 全局异常捕获 =====')
    print(f'异常类型: {type(exception).__name__}')
    print(f'异常信息: {str(exception)}')
    print('异常栈: ')
    print(traceback.format_exc())
    print('=====\\n')
    # 向所有在线客户端推送错误提示
    ui.notify(f'系统异常: {str(exception)}', color='negative', multi=True)
app.on_exception(handle_exception)

# 触发异常的测试按钮
def raise_error():
    # 故意抛出未捕获的异常
    1 / 0 # 除零错误
ui.button('触发测试异常', on_click=raise_error)

ui.run()
```

三、结合示例代码的深度解析（用户提供的 main.py）

用户提供的示例代码核心是通过 `on_connect` 记录最后一个客户端连接的时间，并实时展示在 UI 上，以下拆解关键逻辑：

代码逐行解析

```
from datetime import datetime
from nicegui import app, ui

# 初始化全局变量，记录最后一次连接时间
dt = datetime.now()

# 定义 on_connect 回调函数：每次客户端连接时更新 dt
def handle_connection():
    global dt
    dt = datetime.now()
# 注册回调到 on_connect 事件
```

```
app.on_connect(handle_connection)

# 创建 UI 标签，用于展示时间
label = ui.label()
# 定时器每秒更新标签文本，格式化显示最后一次连接时间
ui.timer(1, lambda: label.set_text(f'Last new connection: {dt:%H:%M:%S}'))

# 启动应用
ui.run()
```

关键特性验证

- 1. 触发时机：每打开一个新的浏览器标签页访问应用，`handle_connection` 会立即执行，`dt` 被更新为当前时间；
- 2. 全局变量共享：`dt` 是全局变量，所有客户端共享该值，因此所有客户端的 UI 都会显示“最后一个连接的客户端”的连接时间；
- 3. 定时器逻辑：`ui.timer(1, ...)` 每秒执行一次 lambda 函数，更新标签文本，实现实时刷新；
- 4. 无参数回调：示例中 `handle_connection` 未接收 `Client` 参数，框架允许省略可选参数，仅执行核心逻辑。

优化建议（增强示例）

若需区分“每个客户端的首次连接时间”，可结合 `Client` 参数和客户端存储（`client.storage`）：

```
from datetime import datetime
from nicegui import app, ui, Client

# 注册带 Client 参数的 on_connect 回调
def handle_connection(client: Client):
    # 将当前客户端的连接时间存入其专属存储（每个客户端独立）
    client.storage['connect_time'] = datetime.now()
    # 向该客户端推送专属提示
    ui.notify(f'你于 {client.storage["connect_time"]:%H:%M:%S} 连接', multi=False)
app.on_connect(handle_connection)

# 展示当前客户端的连接时间
label = ui.label()
ui.timer(1, lambda: label.set_text(
    f'你的连接时间: {ui.client.storage.get("connect_time", "未连接"):%H:%M:%S}'
))

ui.run()
```

四、生命周期事件与通用 Event 的对比

维度	生命周期事件	通用 Event 类
定义方式	框架预定义（ <code>app.on_xxx</code> ），无需手动创建	开发者手动创建（ <code>Event[类型]()</code> ）
触发主体	框架自动触发（如应用启动、客户端连接）	开发者手动触发（ <code>emit()</code> / <code>call()</code> ）

维度	生命周期事件	通用 Event 类
作用范围	全局 / 客户端级（框架层面）	自定义范围（模块间、组件间）
参数规则	固定可选参数（如 <code>client/exception</code> ）	自定义参数（由类型注解限制）
核心用途	应用 / 客户端生命周期管理	模块间解耦通信

五、最佳实践与注意事项

1. 最佳实践

- 资源管理： `on_startup` 初始化全局资源（数据库、缓存）， `on_delete` 清理客户端专属资源， `on_shutdown` 清理全局资源；
- 异常处理：通过 `on_exception` 实现全局异常兜底，避免应用崩溃；
- 客户端隔离：使用 `client.storage` 存储客户端专属数据，避免全局变量污染；
- 异步优先：耗时操作（如 IO、网络请求）建议用异步回调（ `async def` ），避免阻塞应用主线程。

2. 注意事项

- `on_disconnect` 不等于 “客户端永久离线”：需结合 `on_delete` 判断是否彻底清理资源；
- 热重载影响： `on_startup/on_shutdown` 在热重载（ `reload=True` ）时会重复触发，需确保回调幂等（多次执行无副作用）；
- 回调执行顺序：框架不保证多个同类型回调的执行顺序，若有依赖需手动控制；
- 避免阻塞： `on_startup/on_shutdown` 回调若阻塞过久，会影响应用启动 / 关闭速度，耗时操作建议异步执行。

六、总结

NiceGUI 的生命周期事件是框架提供的核心钩子，覆盖应用从启动到关闭、客户端从连接到销毁的全流程，是实现“自动化资源管理、全局状态监控、异常兜底”的关键工具。其核心价值在于：

1. 无需手动监听底层网络 / 进程事件，框架封装后更易用；
2. 区分应用级和客户端级事件，适配不同粒度的逻辑执行；
3. 兼容同步 / 异步回调，适配多样化的业务场景；
4. 与通用 `Event` 类互补，前者聚焦框架生命周期，后者聚焦自定义模块通信。

合理使用生命周期事件，可显著提升 NiceGUI 应用的健壮性、可维护性，尤其适合需要管理多客户端连接、全局资源、异常监控的场景。